Institute of Software Engineering

ITS 2140: Machine Learning -

Group Project Specification

(Semester 4, 2026)

Project Title: Production-Grade E-Commerce Analytics and MLOps System



Part 1: Project Overview and Business Context

1.1 Introduction

This document outlines the final group project for the module **ITS 2140: Machine Learning: Foundations to Production Systems**. This capstone project is designed to serve as a comprehensive, hands-on application of every major concept covered in the syllabus. Students will progress from foundational data analysis and model training to the complex challenges of designing, deploying, and planning for the maintenance of a machine learning system in a production environment.

Table 1: Syllabus Alignment Matrix

Project Phase/Task	Relevant Syllabus Module(s)	Key Concepts Covered	Course Objective(s) Addressed
Task 3.1: Tabular Data Characterization & Preprocessing	Module 2: Foundational Supervised Learning Concepts	Analysis of dataset properties, feature distributions, handling missing values, categorical encoding, scaling of numerical features, and building reproducible preprocessing pipelines using Scikit-learn.	2. Characterize Data & Models; 6. Design End-to-End ML Workflows

Task 3.2: Predictive Regression Modeling	Module 3: Supervised Learning - Regression	Multiple linear regression, loss functions (MSE, MAE), data splits, k-fold cross-validation, stochastic gradient descent, and hyperparameter tuning.	3. Implement Regression Solutions
Task 3.3: Multi-class Classification Systems	Module 4: Supervised Learning - Classification	Multinomial logistic regression, softmax probability outputs, high-dimensional decision boundaries, categorical cross-entropy loss, and decision threshold calibration.	4. Develop Classification Pipelines
Task 3.5: Unsupervised Behavioral Clustering	Module 5: Unsupervised Learning - Clustering	k-Means clustering, cluster quality evaluation (WCSS), centroid initialization, Euclidean similarity, and interpreting clusters for business insights or anomaly detection.	5. Apply Clustering Techniques

Task 4.1, 4.2, 4.3: Production Deployment & MLOps Infrastructure	Module 6: Production ML Systems	System architecture, MLflow experiment tracking, model artifact serialization, cloud storage integration, Vertex AI deployment, pre-built container usage, and RESTful endpoint creation.	6. Design End-to-End ML Workflows
Task 3.4: Rigorous Performance Evaluation	Modules 3 & 4: Supervised Evaluation	Confusion matrix analysis, class-specific precision, recall, F1-score, and contextualizing performance considering business risk implications.	3 & 4. Contextual Evaluation Metrics

1.2 Fictional Scenario: A Strategic Mandate from AuraCart Retail Analytics

For the duration of this capstone project, candidate groups will operate as an external Machine Learning Engineering consultancy contracted by **AuraCart Retail Analytics**. AuraCart is a rapidly scaling, mid-tier digital retail platform currently experiencing severe operational friction due to unprecedented transaction volumes, highly volatile supply chain logistics, and a profoundly heterogeneous customer demographic base. The organization's executive board has identified a critical deficiency in their strategic data utilization architecture: while their systems accumulate massive volumes of transactional data daily, the enterprise currently relies entirely on manual heuristics, intuition-based decision-making, and lagging historical reports to drive dynamic pricing strategies, predict logistical bottlenecks, and execute customer retention campaigns.

The executive mandate assigned to the engineering teams involves the conception, mathematical optimization, and cloud deployment of **a unified, multi-faceted predictive analytics engine**. AuraCart requires an intelligent system capable of executing three distinct, simultaneous algorithmic tasks. First, the platform must utilize continuous regression mechanics to **predict the final monetary value of incoming orders** to facilitate dynamic revenue forecasting and optimize supply chain resource allocation. Second, the system must **automatically classify orders into distinct delivery status categories**, serving as an early-warning mechanism to identify high-risk transactions that possess a high statistical probability of resulting in delayed shipments or costly customer returns, and simultaneously **classify buyers into specific customer segments** to help the marketing team target their campaigns. Third, the marketing and customer success division requires an **unsupervised analytical mechanism to process numerical features and group customers based on latent purchasing behaviors**, enabling hyper-personalized promotional campaigns and mitigating customer churn.

Successfully delivering this mandate requires an engineering solution that extends beyond traditional offline statistical modeling. AuraCart's IT infrastructure requires that all final predictive models be fully integrated into its centralized cloud environment. Engineering teams must use the **MLflow** tracking platform to systematically track and reproduce all algorithmic experiments, hyperparameter configurations, and environment dependencies. Furthermore, the champion models selected from the experimental phase must be formally registered, serialized alongside their preprocessing pipelines, containerized, and deployed as live RESTful inference endpoints on Google Cloud Vertex AI. This deployment allows AuraCart's frontend microservices to programmatically query the models in real-time. The academic evaluation of the final system will penalize architectural weaknesses or incomplete documentation. A model that achieves high offline accuracy but fails to deploy successfully through the Vertex AI pipeline, or fails to accept a properly formatted JSON payload, will be considered an operational failure in this assessment.

Part 2: The Asset: Tabular E-Commerce Data Deep Dive

2.1 Dataset Overview and Provenance

To fulfill the rigorous requirements of the AuraCart mandate, the engineering teams will exclusively utilize the "**E-commerce Customer Order Behavior Dataset**", a high-quality, publicly accessible tabular asset hosted on the Hugging Face hub.

Hugging Face Link for the dataset:
https://huggingface.co/datasets/millat/e-commerce-orders

This dataset is an ideal asset for AuraCart's initiative, as it mirrors the exact type of data the company processes daily. The dataset comprises **10,000 discrete transactional records**.

2.2 Key Data Characteristics and Inherent Challenges

A thorough understanding of the dataset is essential for building high-performing predictive models. The **millat/e-commerce-orders** dataset presents several realistic challenges that reflect operational realities in e-commerce. These challenges are not errors but opportunities for your team to demonstrate robust modeling and engineering strategies.

- **Class Imbalance in Delivery Status:** The delivery status feature is heavily skewed, mirroring real-world e-commerce operations:
    - **Delivered:** ~70%
    - **Shipped:** ~20%
    - **Pending:** ~5%
    - **Returned:** ~5%

This pronounced imbalance can bias naive classification algorithms toward the majority class, as standard loss functions favor the path of least resistance. Your team must implement strategies to counteract this bias, such as:

- Synthetic minority over-sampling (SMOTE)
- Undersampling of the majority class
- Stratified cross-validation to ensure representative folds
- Class-weighted penalties in the objective function

- **Customer Segmentation Distribution:** Similarly, the customer segmentation distribution reflects realistic market dynamics, with approximately 50% constituting 'New' customers, 35% categorized as 'Returning', and a mere 15% classified as high-value 'VIP' clients. Predicting these segments requires nuanced decision boundary modeling. Traditional accuracy metrics may hide poor performance on the critical **VIP cohort**, so your team should consider additional metrics (e.g., F1-score or class-specific precision/recall) for evaluation.
- **Continuous Variables and Feature Scaling:** The dataset contains continuous features such as transaction price, which exhibit varying variance and potentially non-Gaussian distributions. Proper preprocessing is essential to normalize features for gradient descent optimization and prevent features with large ranges from dominating Euclidean distance calculations or weight updates.

The following data dictionary provides a comprehensive reference for each feature in the dataset.

Table 2: Dataset Dictionary

Feature Name	Data Type	Description	How to Handle It in the Project
order_id	string	A unique code for each order.	Drop this column. It is just an ID and won't help the model learn anything useful.
customer_id	int	A unique number identifying the buyer.	Drop this column. It is just an ID.
product_id	int	A unique number identifying the item.	Drop or decide if this specific number is useful for your tasks.

category	string	The department the item belongs to (e.g., Electronics, Clothing, Home, Books, Beauty, Toys).	Convert this text into numbers (like one-hot encoding) so the model can read it.
price	float	The cost of the item ($5.00 to $500.00).	This is your main target to predict for the regression task. Scale the numbers so they aren't too large compared to others.
quantity	int	How many items the customer bought (1 to 10).	Use as a standard feature to help predict other things.
order_date	datetime	The exact date and time the customer placed the order.	You can't use raw dates directly. Extract the month, day, or hour as new number columns.
shipping_date	datetime	The exact date and time the package left the warehouse.	Same as order date. You could also calculate the days between order and shipping.
delivery_status	string	Where the package currently is (Delivered, Shipped, Pending, Returned).	This is your target for the classification task. Convert it to numbers.

payment_method	string	How the customer paid (Credit Card, PayPal, Debit Card, Apple Pay, Google Pay).	Convert this text into numbers.
device_type	string	What screen the customer used (Mobile, Desktop, Tablet).	Convert this text into numbers.
channel	string	How the customer found the website (Organic, Paid Search, Email, Social).	Convert this text into numbers.
shipping_address	string	Where the package is going (Street, City, State, ZIP).	Too messy to use directly as text. Either drop it or extract the state/city if you want to be creative.
billing_address	string	The address linked to their payment method.	Same as shipping address.
customer_segment	string	What kind of buyer they are (New, Returning, VIP).	Convert this to numbers to use as a feature or target.

Part 3: Phase 1 - Foundational Model Development and Analysis

This phase focuses on applying the supervised and unsupervised learning techniques covered in Modules 2 through 5 of the syllabus. The objective is to explore the dataset, perform appropriate preprocessing, and build multiple machine learning models for the given prediction tasks. Your team is expected to carefully analyze the data, experiment with different algorithms, and improve model performance through systematic tuning and evaluation. All data preparation steps, modeling decisions, and evaluation results must be clearly documented and justified.

3.1 Task: Exploratory Data Analysis (EDA) and Preprocessing Pipeline

Before any modeling can begin, a deep understanding of the text data is required. Your team must perform a comprehensive EDA to uncover patterns and understand the characteristics of the dataset.

Requirements:

1. **Analysis of Continuous Variables:** Analyze the distribution of numerical variables, particularly the **transaction price** feature. Use visualizations such as histograms or density plots to observe the shape of the distribution and detect potential skewness or outliers.
2. **Feature Correlation Analysis:** Generate a correlation matrix (using Pearson or Spearman correlation) to examine relationships between numerical features. Highly correlated predictors may introduce redundancy and negatively affect some models. Your team should identify and discuss any strong correlations discovered in the dataset.
3. **Categorical Variable Analysis:** Analyze and visualize the distribution of key categorical variables such as **customer_segment** and **delivery_status**. These visualizations should help determine whether class imbalance exists and provide justification for any sampling or modeling strategies used later.
4. **Key Findings Summary:** Summarize the important insights discovered during the EDA process. For example, highlight significant correlations, unusual distributions, or severe class imbalances that could influence modeling decisions.

Preprocessing Pipeline using Scikit-learn:

To ensure consistency and reproducibility, your team must build a **data preprocessing pipeline** using the Scikit-learn Pipeline framework. This pipeline should combine all preprocessing steps into a single reusable workflow so that the same transformations are applied during training, validation, and deployment. Your pipeline should address the following preprocessing tasks:

1. **Categorical Encoding:** Convert categorical variables into numerical form using appropriate encoding techniques such as:
    - **One-hot encoding** for nominal categories without inherent order
    - **Ordinal encoding** where meaningful ordering exists
2. **Feature Scaling:** Scaling ensures that features with larger numeric ranges do not dominate the model training process. Apply feature scaling to numerical variables using techniques such as:
    - Standardization (z-score normalization)
    - Min–max normalization

The final preprocessing pipeline should be modular and reusable so that it can be easily integrated into the later stages of the project, including model training, evaluation, and deployment. The pipeline should also be compatible with experiment tracking and model management tools used in later phases.

3.2 Task: Continuous Price Prediction Modeling

After preparing and cleaning the dataset, the next step is to build a **predictive model that can estimate the price of an incoming order** using the available input features.

Requirements:

1. **Implement a Multiple Linear Regression Model:** Implement a Multiple Linear Regression model to predict the **continuous price value** of an order. This model learns how different input variables (features) influence the final price by finding the best

mathematical relationship between them. The regression model will serve as the **core prediction component** of the AuraCart pricing system. You must:

- Train the model using the prepared training dataset.
- Configure and experiment with the **gradient descent training process**. This includes adjusting important parameters such as:
    - **Learning rate** – controls how large each update step is during model training.
    - **Batch size** – determines how many samples are processed before updating the model.
    - **Epochs** – the number of times the model goes through the full training dataset.

- Clearly explain how changing these parameters affects the learning behavior and performance of the model.

2. **Evaluate the Regression Model:** Evaluate the performance of the regression model using commonly used regression evaluation metrics. You must:
    - Calculate **Mean Squared Error (MSE)**.
    - Calculate **Mean Absolute Error (MAE)**.
    - Explain the difference between these two metrics and discuss how large prediction errors may impact financial decision-making in the AuraCart system.

3. **Use Cross-Validation for Reliable Evaluation:** To obtain a more reliable estimate of model performance, you must:
    - Apply **k-fold cross-validation** when evaluating the regression model.
    - Analyze the results obtained from cross-validation.
    - Discuss whether the model shows signs of **underfitting (high bias)** or **overfitting (high variance)** and explain possible reasons.

3.3 Task: Multi-class Classification Modeling

The second prediction task focuses on **classifying transactional events into predefined categories**. The model should predict categorical outputs such as **customer_segment** and **delivery_status** using the available features in the dataset.

Requirements:

1. **Implement a Softmax Regression Model:** Implement a Softmax Regression model (also known as Multinomial Logistic Regression) to perform multi-class classification. This model is designed to handle problems where the target variable contains more than two possible classes. It converts model outputs into probabilities and selects the most likely class as the prediction. You must:
    - Train the classification model using the prepared dataset.
    - Explain why **linear regression models are not suitable for predicting categorical outcomes**.
    - Describe how the **Softmax function converts model outputs into probabilities across multiple classes**.
    - Use **log-loss (also known as categorical cross-entropy)** during training and explain how this loss function measures the difference between predicted probabilities and the true class labels.

2. **Analyze and Adjust Decision Thresholds:** In many real-world datasets, some classes occur much less frequently than others. To handle this situation, you must:
    - Extract the **predicted probabilities** produced by the Softmax model.
    - Experiment with **different decision thresholds** instead of always choosing the class with the highest probability.
    - Analyze how changing these thresholds can help improve the detection of **rare but important classes** (for example, predicting a rare “Returned” delivery status).
    - Explain how threshold adjustments change the **sensitivity of the classifier**, especially when working with imbalanced datasets.

3. **Evaluate the Classification Model:** To properly evaluate the classification model, you must go beyond simple accuracy measures. You must:
   - Generate and analyze a **confusion matrix**.
   - Examine the **types of misclassification** made by the model between different classes.
   - Provide a clear explanation of what these errors mean in the context of the business problem.

3.4 Task: Classification Model Performance Analysis and Risk Evaluation

After building the classification model, the next step is to **carefully analyze how well the model performs across different classes**. In many real-world datasets, some classes appear much more frequently than others. Because of this, relying only on **overall accuracy** can give a misleading impression of model performance.

Requirements:

1. **Generate and Analyze the Confusion Matrix:** To understand how the model behaves for each class, teams must analyze the **confusion matrix** produced by the classification model. You must:
   ○ Generate and clearly visualize the confusion matrix.
   ○ Examine how predictions are distributed across the different classes.
   ○ Identify common **misclassification patterns**, showing which classes are often confused with one another.

2. **Calculate Class-wise Performance Metrics:** Model performance must be analyzed **separately for each class** rather than relying only on a single overall metric. You must:
   ○ Calculate **Precision**, **Recall**, and **F1-score** for every class.
   ○ Explain what each metric means in the context of classification performance.
   ○ Discuss how these metrics help reveal weaknesses that may not be visible when looking only at accuracy.

3. **Discuss Precision–Recall Trade-offs in the Business Context:** The performance analysis must also consider the **operational impact of prediction errors** within the AuraCart system. You must:
   ○ Explain the trade-off between **precision and recall** for important prediction tasks.
   ○ Use examples from the dataset (such as predicting the **"Returned" delivery status**) to illustrate these trade-offs.
   ○ Discuss the business consequences of different types of errors.

4. **Analyze Asymmetric Risk in Decision Making:** In real business environments, different prediction errors can have **different financial consequences**. You must:
   ○ Compare the impact of **false positives** and **false negatives** for key prediction scenarios.
   ○ For example, discuss whether the business loses more when the system incorrectly predicts a return (false positive) or when it fails to predict an actual return (false negative).
   ○ Justify the final model choice by explaining how it balances these risks and supports better business decision-making.

3.5 Task: Unsupervised Learning – Customer Behavior Clustering

In this task, your team will apply **unsupervised learning** to discover hidden patterns in customer behavior. The goal is to group similar transactions or customers together based on their characteristics, without using any predefined labels.

Requirements:

1. **Clustering with k-Means:** Apply the K-means Clustering algorithm to the dataset after preprocessing and feature scaling.
2. **Selecting the Number of Clusters (k):** The number of clusters cannot be chosen randomly. Your team must determine a suitable value for k using quantitative methods such as:
   - The **Elbow Method**, by plotting the Within-Cluster Sum of Squares (WCSS)

- The **Silhouette Score**, which measures cluster separation and cohesion

Provide a clear explanation and justification for the value of **k** you select.

3. **Cluster Interpretation:** After clustering is completed, analyze the resulting clusters by examining the characteristics of their centroids (cluster centers). Identify what makes each cluster unique based on the dominant feature values. For example, you might identify clusters such as:
    - Frequent low-value buyers
    - High-value VIP customers
    - Customers with frequent returns
    - Occasional buyers with high transaction values
4. **Business Insights:** Translate the discovered clusters into meaningful business insights for **AuraCart**. Explain how these customer segments could support business strategies such as:
    - Targeted marketing campaigns
    - Personalized promotions
    - Dynamic pricing strategies
    - Inventory planning and recommendations

Part 4: Phase 2 – Deploying the Production System with MLflow and Vertex AI

In this phase, your team will transform the models developed in Phase 1 into a **production-ready machine learning system**. The goal is to apply modern **MLOps practices** by tracking experiments with MLflow and deploying the final model as a live prediction service using Vertex AI. This phase evaluates your ability to manage experiments, package models, and deploy a scalable cloud-based inference service.

4.1 Task: Experiment Tracking with MLflow

During Phase 1, your team will train multiple models and test different hyperparameter configurations. **All experiments** must be tracked systematically using the **MLflow tracking**

platform.

Requirements

1. **Integrate MLflow into Training Code:** Modify your training scripts to log experiments using the MLflow Python API. You may use manual logging or automatic logging features such as **mlflow.sklearn.autolog()** when using Scikit-learn models.
2. **Track Model Experiments:** For each training run, MLflow should record:
    - Model parameters (e.g., learning rate, number of epochs, solver)
    - Evaluation metrics (e.g., accuracy, F1-score, MSE, MAE, number of clusters)
    - Model artifacts and preprocessing pipelines
    - Environment information and dependencies
3. **Experiment Comparison:** Use the MLflow Tracking UI to compare different runs and identify the best-performing models.
4. **Documentation:** Your report must include screenshots of the MLflow interface showing multiple experiment runs and the selected best model.

4.2 Task: Model Packaging and Artifact Management

To simplify the deployment phase, **only one model will be deployed**. Teams must deploy the final model trained for the **customer segment prediction task** developed in **Section 3.3 (Multi-class Classification Modeling)**.

Requirements

1. **Select the Final Model:** Using the experiment results tracked in MLflow, identify the best-performing model for predicting **customer_segment**.
2. **Create a Unified Prediction Pipeline:** Combine the full preprocessing pipeline and the trained classification model into a single **Scikit-learn Pipeline object**. This ensures that all preprocessing steps and predictions are executed consistently during inference.
3. **Save the Model Artifact:** The trained model artifact logged by MLflow may be stored as a .pkl file. For deployment compatibility (i.e., to use Vertex AI's pre-built container

images), teams should serialize the final pipeline using the Joblib library and save it as: **model.joblib**

4. **Capture Dependencies:** Create a requirements.txt file listing all Python libraries required to run the model (e.g., Scikit-learn, pandas, numpy).
5. **Upload Artifacts to Google Cloud Storage:** Upload the following files to a new bucket in Google Cloud Storage (GCS). These files will be used during the deployment stage:
    - model.joblib
    - requirements.txt

4.3 Task: Deploying a Live Prediction Service with Vertex AI

In this task, your trained **customer segment prediction model** will be deployed as a **live REST API service** using Vertex AI.

Requirements

1. **Import Model into Vertex AI:** Import the model stored in your Google Cloud Storage bucket into the **Vertex AI Model Registry**.
2. **Select a Pre-built Prediction Container:** Choose the **Scikit-learn pre-built prediction container** provided by Vertex AI. This container automatically provides the runtime environment and HTTP server needed to serve predictions.
3. **Deploy the Model to an Endpoint:** Deploy the registered model to a new **Vertex AI Endpoint**. Vertex AI will automatically manage the infrastructure required to host the model.
4. **Test the Live Endpoint:** Send a prediction request to your deployed endpoint using a JSON payload representing a new e-commerce transaction. The endpoint should return a prediction indicating the **customer segment classification**.
5. **Verification:** Your final report must include:
    - Screenshot of the deployed endpoint
    - Screenshot or output of a successful prediction response
    - A short explanation of the deployment steps

Part 5: Final Deliverables and Reporting Structure

Your team's work will be assessed based on three key deliverables, each designed to evaluate a different aspect of your project execution: the technical report, the codebase, and the final presentation.

5.1 The Comprehensive Technical Report (PDF format)

This report is the primary written deliverable and should be structured as a professional document for the executive leadership of AuraCart and peer machine learning architects. It should be clear, concise, and well-supported by data and visualizations. The main body of your report must be between **2,000 and 2,500 words**. This is a strict requirement. The word count does not include the title page, table of contents, or appendix.

Required Sections:

1. **Executive Summary:** Provide a brief, high-level summary intended for a non-technical manager at AuraCart. This section should outline the business objectives, the machine learning approaches used, the key performance results of the final models, and the potential business value of the system.
2. **Introduction and Business Context:** Introduce the project and explain the business problem AuraCart aims to address. Describe the objectives of the machine learning system, the main analytical tasks involved (regression, classification, and clustering), and provide a short overview of the dataset used.
3. **Data Exploration and Preprocessing:** Summarize the key findings from the exploratory data analysis (EDA), supported by relevant visualizations. Explain the preprocessing steps applied to the dataset, including handling missing values, encoding categorical variables, scaling numerical features, addressing class imbalance, and constructing the preprocessing pipeline using Scikit-learn.
4. **Regression and Classification Modeling:** Describe the supervised learning models developed for the project. Explain the algorithms used, the training process, and the hyperparameter tuning approach. This section should also describe how experiments and model runs were tracked using MLflow.
5. **Model Evaluation and Performance Analysis:** Present the evaluation results of the

trained models using appropriate metrics such as accuracy, precision, recall, F1-score, MAE, or MSE. Include confusion matrices where relevant and explain how the results relate to the business goals of AuraCart.

6. **Customer Behavior Clustering:** Explain how unsupervised learning was used to discover patterns in customer behavior. Describe how the K-means Clustering algorithm was applied, how the number of clusters was selected, and how the resulting clusters can provide useful business insights.
7. **Production Deployment and MLOps Workflow:** Describe how the final model was prepared for deployment. Explain how experiments were tracked with MLflow, how the model artifact was stored in Google Cloud Storage, and how the model was deployed as a prediction service using Vertex AI. Include evidence of a successful prediction request to the deployed endpoint.
8. **Conclusion and Future Work:** Summarize the main outcomes of the project and highlight the key achievements. Discuss any limitations of the current system and suggest possible improvements or future enhancements.

5.2 The Project Codebase (ZIP archive)

Your code should be well-organized, commented, and reproducible. The repository structure should follow best practices. It should contain the files listed below. Don't add any additional files, as that would make things more complex.

/notebooks: This directory will contain separate, numbered notebooks for each distinct phase of the project:

- **1_eda_and_preprocessing.ipynb**: Encapsulates all statistical visualizations, feature engineering logic, correlation analyses, and the final serialization of the Scikit-learn preprocessing pipeline objects.
- **2_supervised_modeling.ipynb**: Contains the MLflow-tracked experiments for both the continuous regression and multi-class classification tasks, including all cross-validation loops, hyperparameter tuning grids, and threshold calibrations.
- **3_unsupervised_clustering.ipynb**: Houses the k-Means implementation, the

algorithmic determination of ‘k’ (elbow plots), and the extraction and analysis of the geometric centroids.

- **4_mlops_deployment.ipynb**: Documents the programmatic interaction with the Google Cloud SDK, detailing the transfer of artifacts from the MLflow registry to GCS, and the API calls utilized to instantiate the Vertex AI Endpoint.

/artifacts: A directory exclusively containing the final, unified pipeline binaries (e.g., .pkl and .joblib files) and the environment requirements.txt file utilized during the cloud deployment.

Requirements:

- **Well-Documented Code:** Code cells must be accompanied by Markdown cells that explain the rationale behind your actions. Do not just write code; explain *why* you are performing a certain step, what choices you made, and how you interpret the output of a code cell.
- **Reproducibility:** The notebooks should run from top to bottom without errors.

Part 6: Evaluation Criteria and Grading Rubric

Evaluation will be based on the quality, depth, and professionalism of all deliverables. The following rubric provides a detailed breakdown of the grading criteria to ensure transparency and fairness.

Table 3: Detailed Grading Rubric

Category	Weight	Criteria for Excellence (A)	Criteria for Competence (B)	Criteria for Satisfactory (C)
Data Analysis & Preprocessing	20%	Comprehensive text EDA with deep insights. Sophisticated and	Good EDA is performed with clear visualizations.	Basic EDA is completed. The preprocessing

		well-justified text preprocessing and vectorization pipeline that robustly handles the challenges of unstructured medical text.	Standard text preprocessing steps are correctly implemented. Major challenges are addressed, but justification may be less thorough.	pipeline is functional but may be overly simplistic. Text data challenges are handled with minimal analysis or justification.
Modeling & Evaluation	30%	The model is correctly implemented and tuned. Evaluation is exceptionally thorough, with a nuanced, context-aware analysis of per-class performance that connects it to the business impact.	The model is implemented correctly. Evaluation uses all required metrics. The performance analysis is justified by the metrics but may lack a deep connection to the business problem.	The model is implemented correctly. Basic evaluation metrics are reported. The performance analysis is based on a single metric (e.g., accuracy) without sufficient context.
System Design & Deployment	30%	The model is successfully deployed to a live Vertex AI endpoint. The use	The model is successfully deployed to a functional Vertex AI endpoint. The	The model is deployed, but the process may contain errors or is

		of a Scikit-learn Pipeline to package the model is correctly implemented and well-explained. The report provides a clear, professional, and easily reproducible step-by-step guide with excellent documentation and screenshots.	documentation in the report is complete but may lack clarity or professional polish in some areas. The Pipeline is used, but its role may not be thoroughly explained.	poorly documented. The endpoint may be functional, but the steps to reproduce it are unclear. The Pipeline object may be implemented incorrectly.
Report & Communication	20%	The technical report is professionally written, well-structured, and easy to follow. All sections are complete and detailed. The presentation is clear, engaging, and effectively	The report is well-written and covers all required sections. The presentation is clear and well-organized. The codebase is functional but may lack extensive comments or a	The report has significant grammatical or structural issues. The presentation is disorganized or fails to communicate key findings. The codebase is difficult to run or

		communicates the project's value to a mixed audience. The codebase is clean, well-documented, and easily reproducible.	clear structure.	understand.